Scaling Graph Chain-of-Thought Reasoning: A Multi-Agent Framework with Efficient LLM Serving

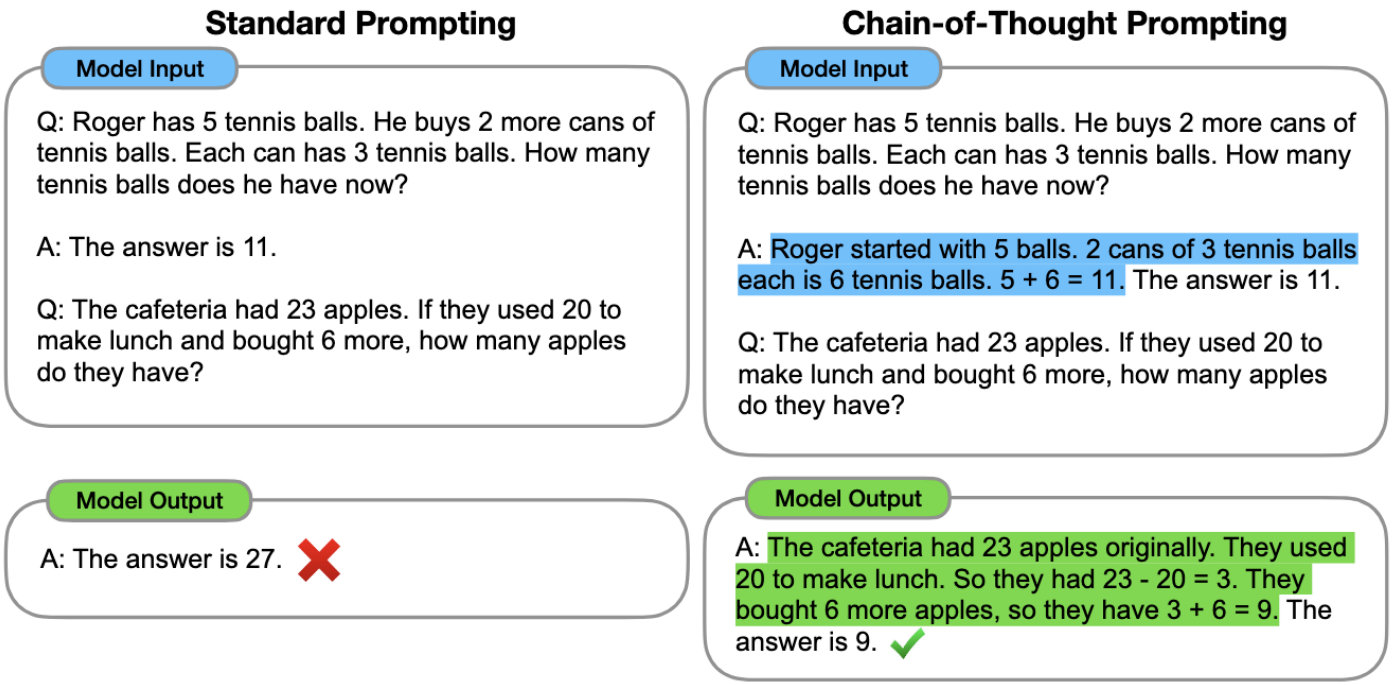
Chengying Huan, Ziheng Meng, Shipeng Li, et al.

January 18, 2026

Outline

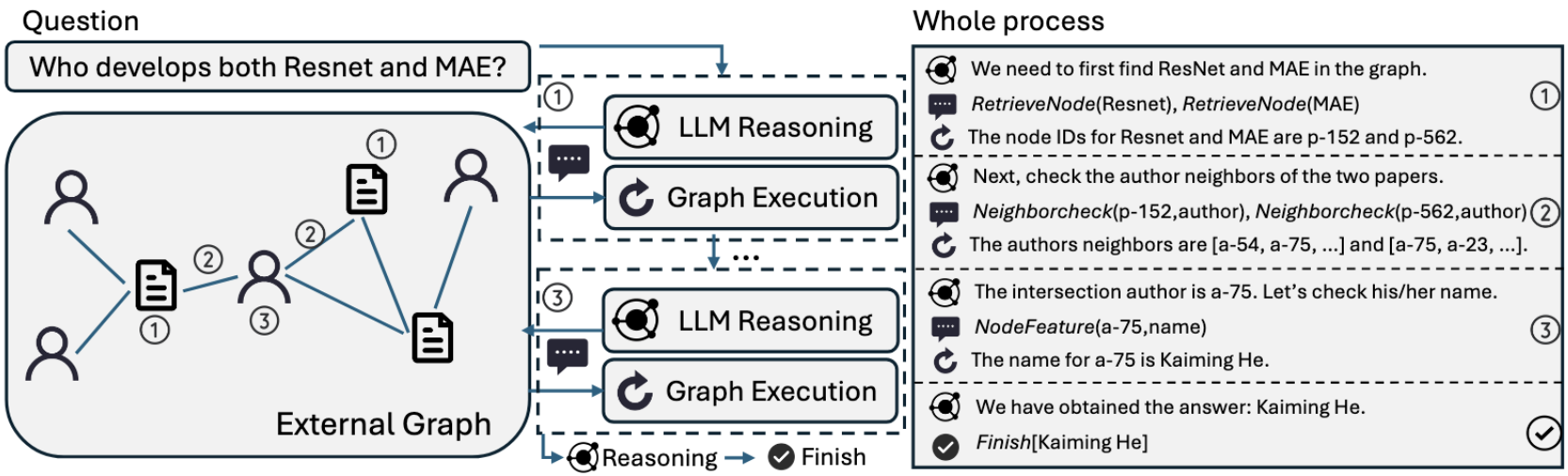
CoT

Chain-of-thought prompting enables large language models to tackle complex arithmetic.



Graph-CoT

- Graph Chain-of-Thought (Graph-CoT) reasoning extends RAG by tightly integrating LLM reasoning with graph retrieval
  - Reasoning with LLMs
  - Interaction between LLMs and Graphs.
  - Execution on Graphs



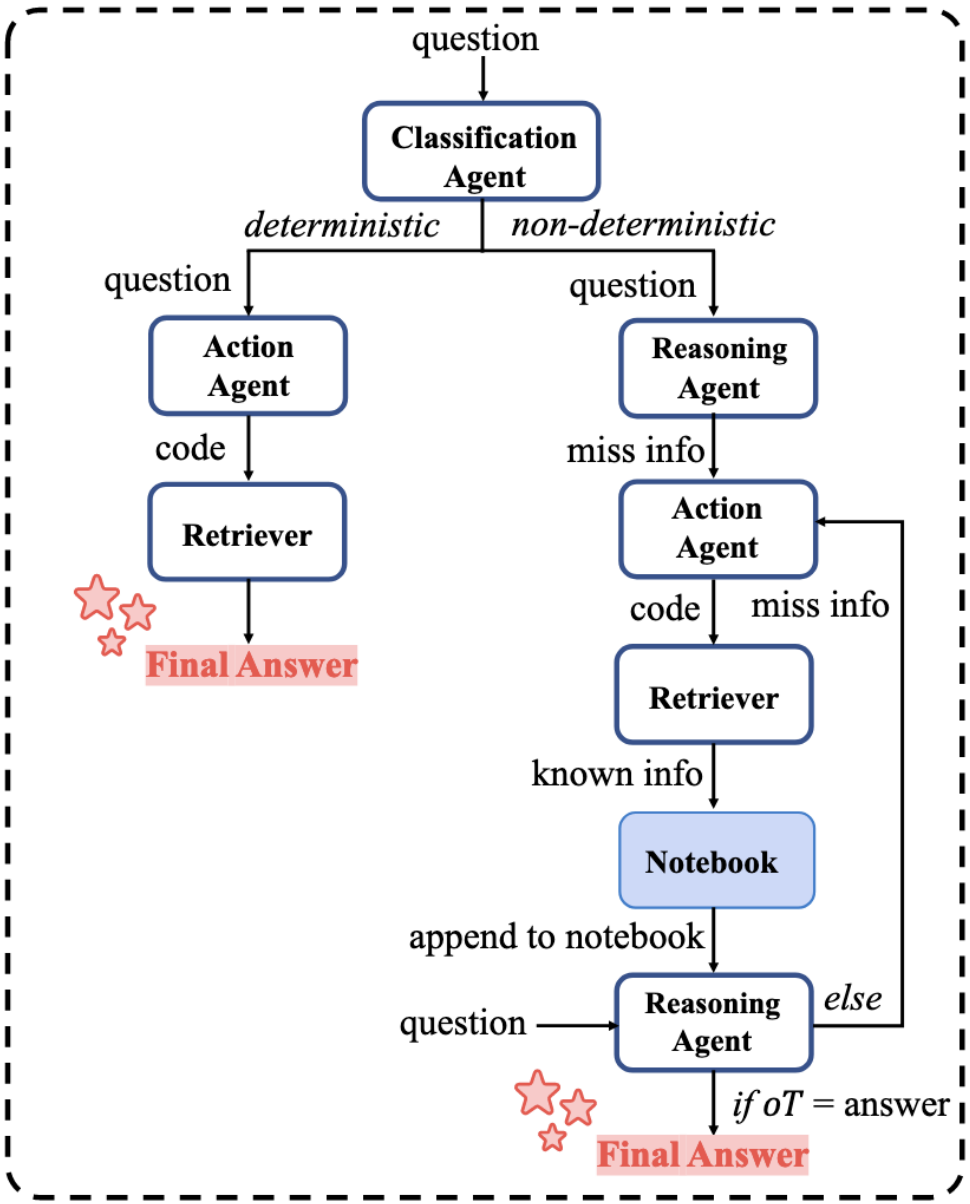
Challenges

- Single-agent limitations.
  - Input sequences grow longer.
  - Accumulate redundant context.
  - High token overhead.
- Inference inefficiencies.
  - End-to-end latencies up to 39s per query.
  - Modest KV-cache hit rates.
  - Retriever latency increases with graph size.

Multi-Agent Graph-CoT Framework

Framework Overview

- Graph RAG Retriever.
- Classification Agent (C-Agent).
- Reasoning Agent (R-Agent).
- Action Agent (A-Agent).



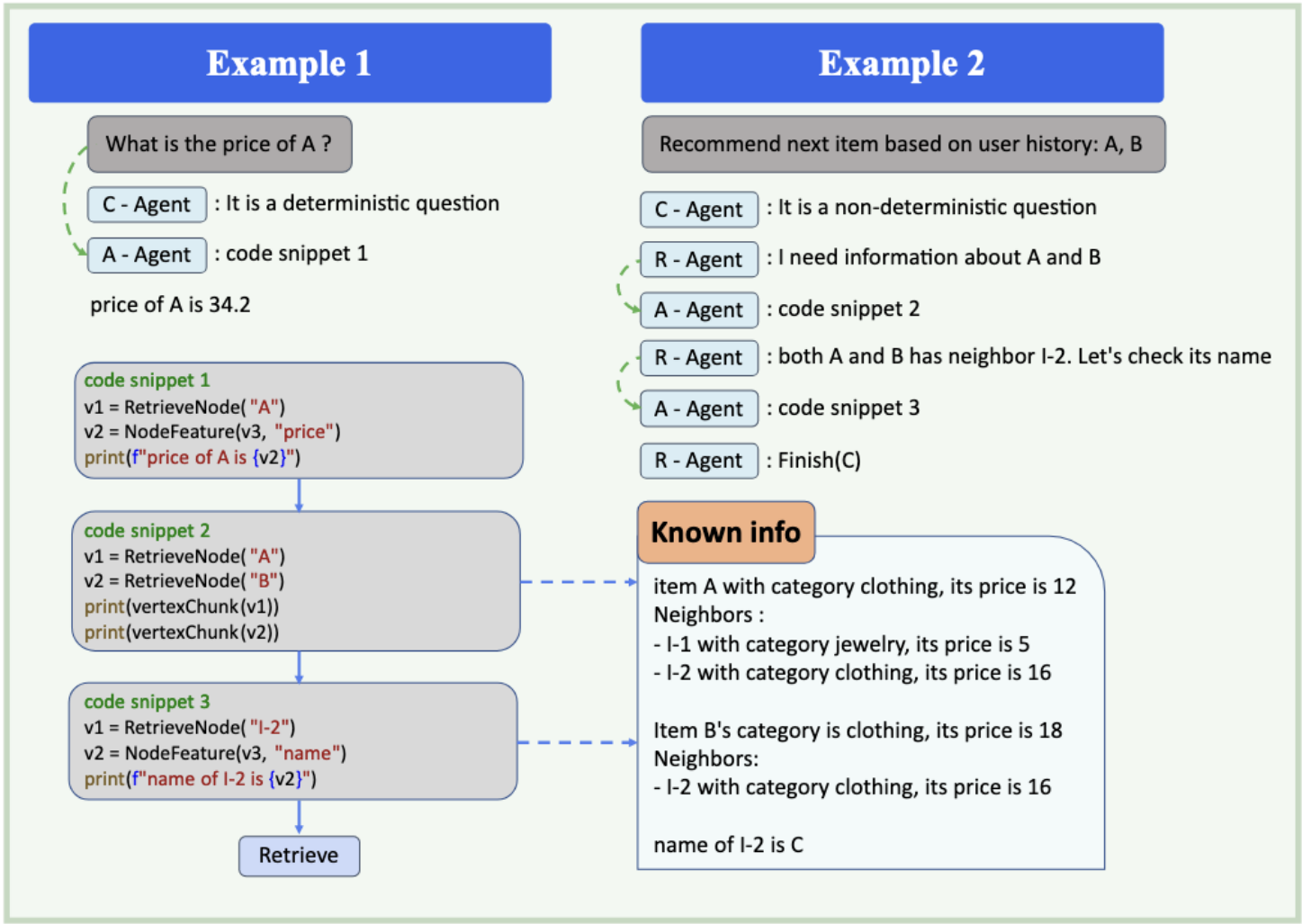
Multi-Agent Reasoning Workflow

Algorithm 1 Multi-Agent Reasoning Workflow.

Require: Question q, C-Agent, R-agent, A-Agent, Retriever, LLM service.

$o_c \leftarrow LLM(P_c(q))$
if $o_c ==$ yes **then**
    $o_a \leftarrow LLM(P_a(q))$
    $a \leftarrow Retriever(o_a)$
else
    Initialize notebook $n \leftarrow \{\}$
    while $True$ **do**
        $o_t \leftarrow LLM(P_t(n, q))$
        if o_t indicates answer completeness **then**
            $a \leftarrow o_t$
            break
        else
            $o_a \leftarrow LLM(P_a(o_t))$
            $o_r \leftarrow Retriever(o_a)$
            $n \leftarrow n \cup \{o_r\}$
        end if
    end while
end if
return final answer a

Multi-agent reasoning example



Multi-agent reasoning Module

Graph CoT

Prompt template
- graph definition
- cot instruction
- graph function
- cot example
- question
- context

Output
- thought / action

C-Agent

Prompt template
- graph definition
- classify instruction
- question

Output
- yes / no

R-Agent

Prompt template
- graph definition
- thought instruction
- known information
- question

Output
- miss information

A-Agent

Prompt template
- graph definition
- action instruction
- graph function
- action example
- miss information

Output
- code snippet

Function	**Input**	**Output**	**Function Description**
RetrieveNode	text	nodeID	Returns the ID of the node most relevant to the input text using vector similarity.
NodeInfo	nodeIDs	vertex chunk	Returns the vertex chunk for the given node.
NodeFeature	nodeIDs, featureName	values	Retrieves feature values for the given nodes.
NodeDegree	nodeID, neighbourType	degree	Returns the number of neighbours of the specified type for the given node.
neighbourCheck	nodeID, neighbourType	nodeIDs	Returns the neighbours of the specified type for the given node.

Code Generation Optimization

Graph-CoT Code Generation

- Select predefined function and provide parameters.

GLM Code Generation

- Synthesize complete executable Python snippets.
- Basic control(e.g., if-else, for).
- Standard data types(set, list, dict).
- Output via print().

8 LLM invocations in Graph-CoT

```
Question : What is the average price of items commonly viewed with A?
Thought 1:  First, I need to find the node for A
Action 1: RetrieveNode[A]
Observation 1: The ID of this retrieval target node is I-1.
Thought 2: With A's node ID, I can find items often viewed together.
Action 2: NeighbourCheck[I-1, also_viewed_item]
Observation 2: The also_viewed_item neighbors of I-1 are: ["I-2", "I-3", "I-4"].
Thought 3: I'll fetch the prices of the "also viewed" items to see the average cost.
Action 3: NodeFeature[I-2, price], NodeFeature[I-3, price], NodeFeature[I-4, price]
Observation 3: The price feature of I-2 are: 1. The price feature of I-2 are: 2.
The price feature of I-2 are: 3.
Thought 4: I'll use the average price of also_viewed items as the typical cost.
Action 4: Finish[2]
```

Replaced by

1 code snippet in GLM (Ours)

```
Question : What is the average price of items commonly viewed with A?
Action : v1 = RetrieveNode("A")
         v2 = NeighbourCheck(v1, "also_viewed_item")
         prices = []
         for v3 in v2:
             v4 = NodeFeature(v3, "price")
             prices.append(v4)
         average = sum(prices) / len(prices)
         print(average)
Finish : 2
```

Token Usage Analysis

Define.

- T_G, T_c, T_a, and T_t denote the average token consumption.
- $T_c \ll T_G$, approximate $T_a + T_t \approx T_G$

Graph-CoT. k_1 steps per question, the total token consumption is $2k_1 \cdot T_G$.

GLM. The token cost:

- *Deterministic*: $T_c + T_a < T_a + T_t \approx T_G$.
- *Non-deterministic*: action agent k_2, reasoning agent $k_2 + 1$. total cost $k_2 \cdot T_a + (k_2 + 1) \cdot T_t < (k_2 + 1) \cdot (T_a + T_t) \approx (k_2 + 1) \cdot T_G$.

Token Reduction. Compared to Graph-CoT:

- Deterministic: $(2k_1 - 1) \cdot T_G$.
- Non-deterministic: $(2k_1 - k_2 - 1) \cdot T_G$.

Graph-CoT-Aware LLM Inference

KV Cache:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

$$\text{token}_1 \rightarrow \text{token}_2 \rightarrow \text{token}_3 \rightarrow \cdots \rightarrow \text{token}_T$$

- KV-Cache automatically implemented during the decoding process of Transformer-based LLM.

RAG with KV Cache.

Retrieved Context

↓

Prompt Assembly

↓

`LLM.generate()`

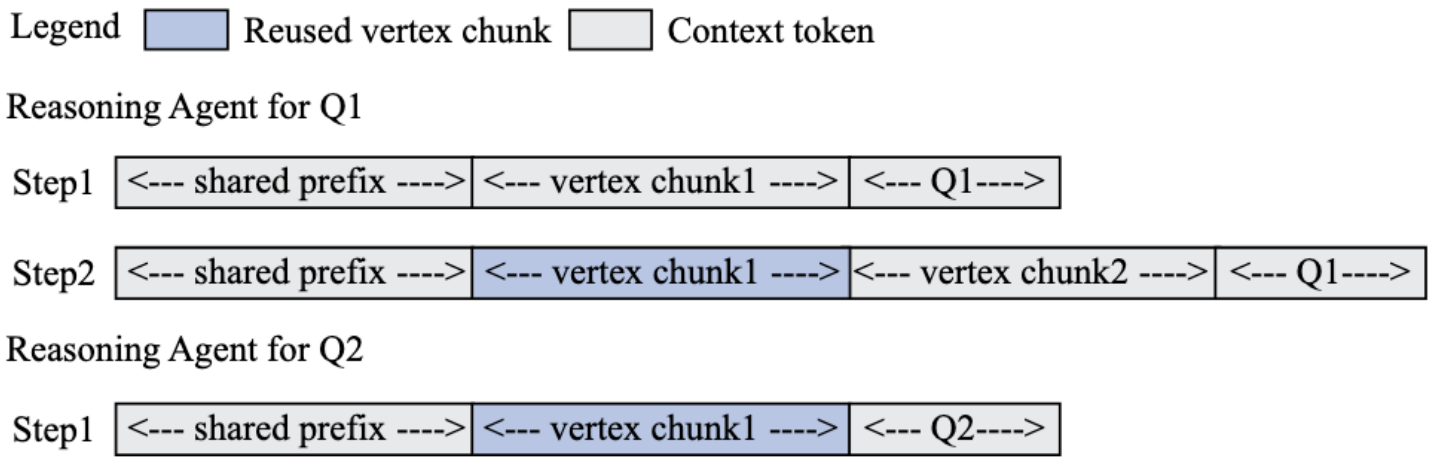
- Same prefix prompt.
- Consistent data ordering (v1, v2, v5, v7).
- Chunked data.

Vertex-Centric KV Cache Reuse Model

Fine-grained retrieval unit: vertex chunk

```
[Node:<nodeID> {k1:v1, k2:v2, ...}]
[neighbours:(n1 {k1:v1, ...}), (n2 {k1:v1, ...}), ...]
```

- Sort vertex's neighbours by weight(e.g., node degree).
- Retain the top-k nodes with highest weight.

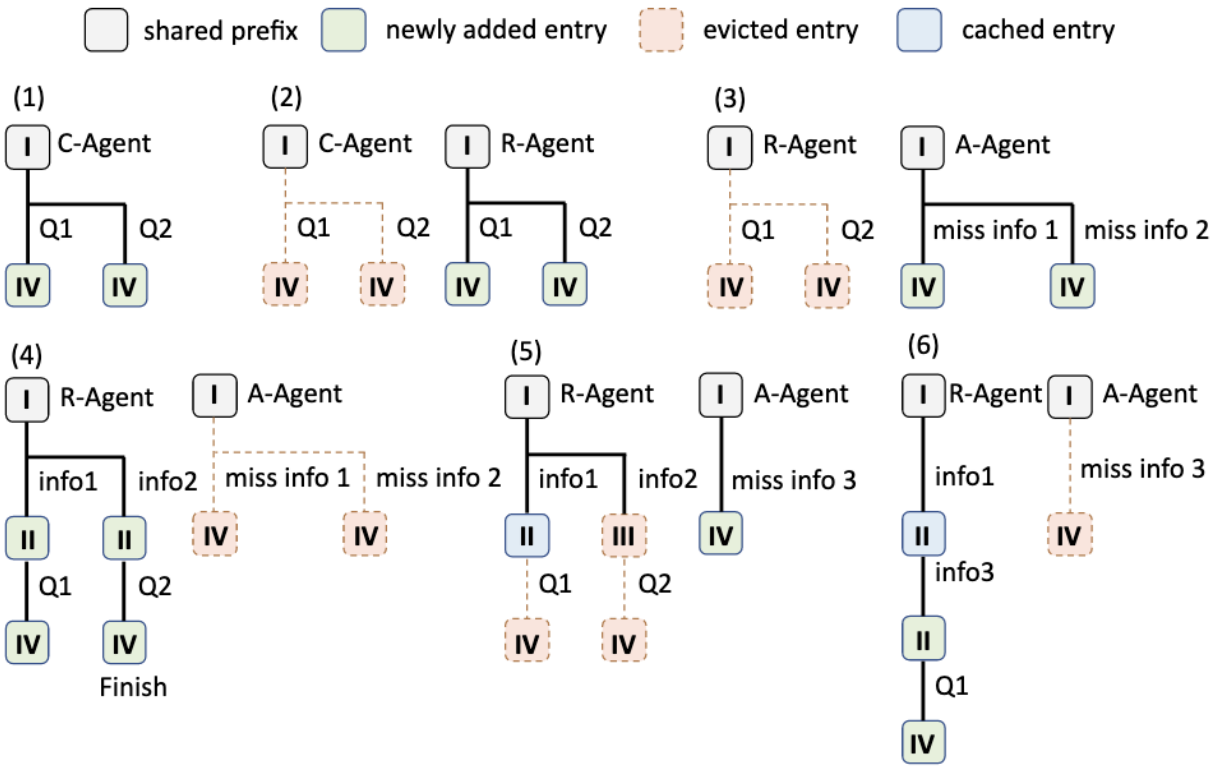


Priority-based KV Cache Eviction Policy

Priority-based KV cache eviction policy:

Maintains multiple queues and evicts the lowest-priority entries first.

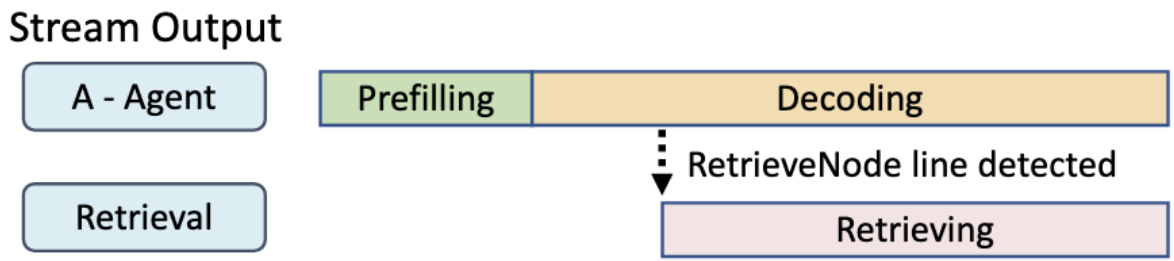
- Priority I: prompt template.
- Priority II: notebook-related KV entries.
- Priority III: after the question, downgrade the notebook content to priority III.
- Priority IV: missed information, e.g.



Pipelined Execution Strategy

Pipelined Execution Strategy:

- the prefill stage and the decoding stage of the line containing the RetrieveNode call.
- the decoding stage of the remaining tokens.



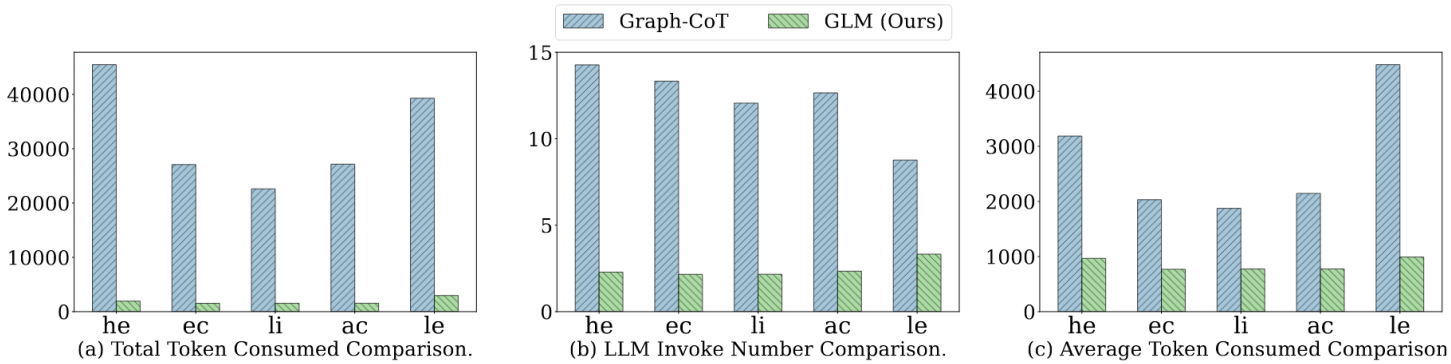
- maintain a global cache that maps RetrieveNode inputs to corresponding output NodeIDs.

Benchmark accuracy comparison

Table 4: Benchmark accuracy comparison of LLM variants evaluated with Rouge-L (R-L) and GPTScore.

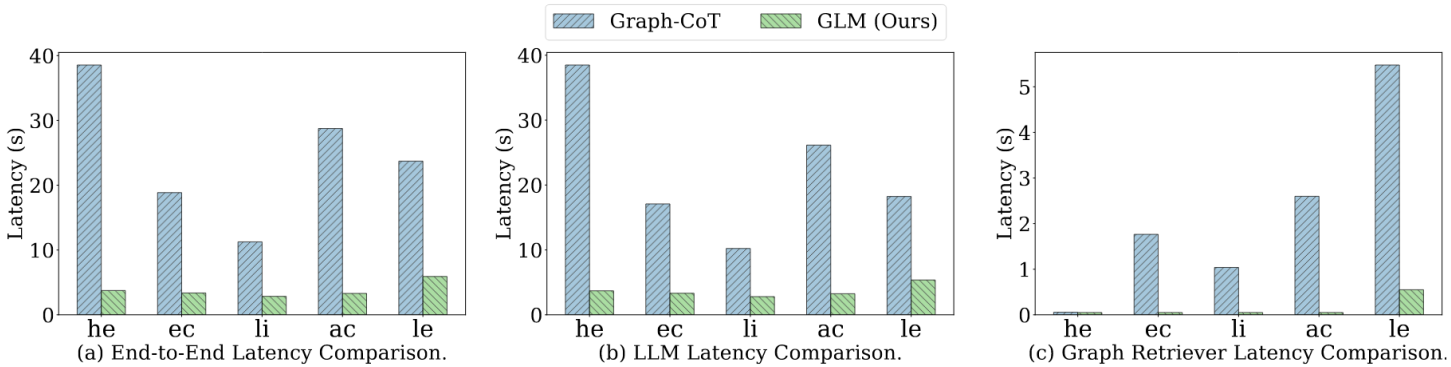
	Academic		E-commerce		Literature		Healthcare		Legal	
	R-L	GPTScore	R-L	GPTScore	R-L	GPTScore	R-L	GPTScore	R-L	GPTScore
Base	0.09	0.15	0.14	0.20	0.11	0.23	0.06	0.20	0.16	0.26
Text RAG	0.09	0.14	0.25	0.31	0.15	0.28	0.04	0.20	0.21	0.26
Graph RAG	0.28	0.33	0.34	0.37	0.21	0.32	0.11	0.16	0.21	0.24
Graph-CoT	0.31	0.4	0.39	0.44	0.42	0.53	0.33	0.46	0.46	0.53
GLM	**0.55**	**0.55**	**0.77**	**0.79**	**0.65**	**0.68**	**0.62**	**0.70**	**0.63**	**0.64**

Token and End-to-end latency comparison



(a) Total Token Consumed Comparison. (b) LLM Invoke Number Comparison. (c) Average Token Consumed Comparison

Figure 10: Token consumption comparison.



(a) End-to-End Latency Comparison. (b) LLM Latency Comparison. (c) Graph Retriever Latency Comparison.

Figure 11: End-to-end latency comparison.

Error Analysis of Code Generation

Analyze failure cases in Graph-CoT and GLM:

categorizes errors into four types:

- unexpected agent output.
- retrieval process error.
- code execution error.
- step limit exceeded.

Table 5: Fine-grained error taxonomy contrasting Graph-CoT and GLM.

Error category	Brief description	Graph-CoT	Graph-CoT with code generation	GLM
Unexpected agent output	The agent violates expected format.	42 (41%)	49 (51%)	2 (4%)
Retrieval process error	The system fetches irrelevant information.	20 (19%)	20 (21%)	18 (43%)
Code execution error	Generated code fails during runtime.	2 (2%)	20 (21%)	20 (48%)
Step limit exceeded	The reasoning exceeds the preset step limit.	38 (37%)	8 (8%)	2 (4%)